Project 5: Recognition using Deep Networks

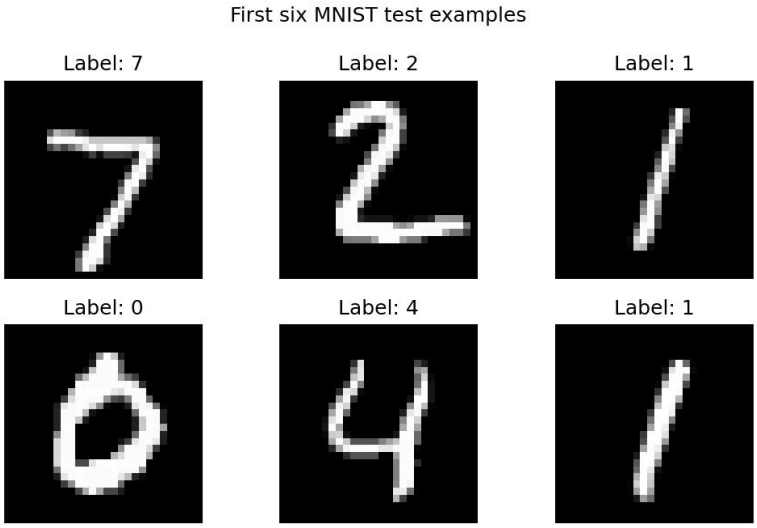
Name: Shreyas Sai Raman

Overall Description:

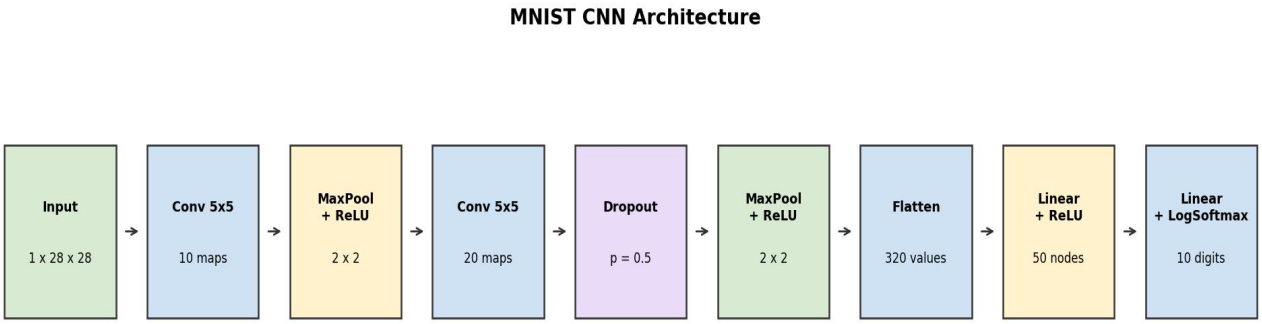
This project explored digit and symbol recognition using deep neural networks. I first built and trained a convolutional neural network on the MNIST handwritten digit dataset, saved trained model, and reloaded it for testing. I then used the network to classify both standard MNIST test images and my own handwritten digit images. After training, I examined the first convolutional layer to understand what features the network learned and visualized how those filters respond to an input digit. I also reused the MNIST network for transfer learning on Greek letters by freezing the pretrained layers and replacing the final classifier. Finally, I implemented transformer-based MNIST models and ran architecture experiments to study how network design choices affect accuracy and training time.

Task 1: MNIST CNN Training

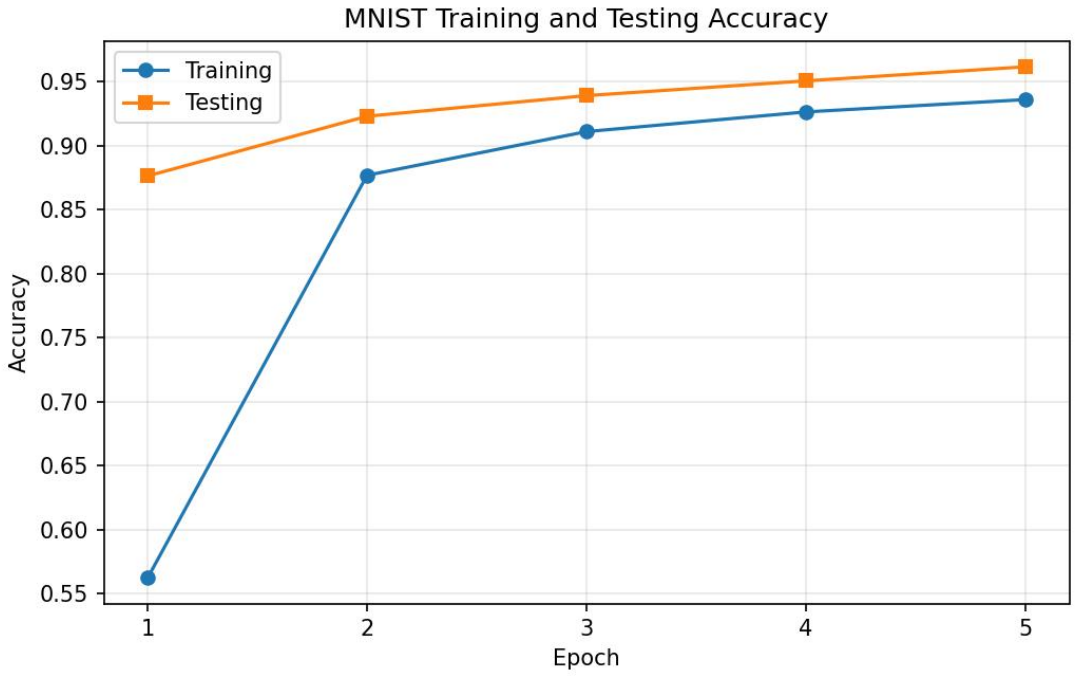
The first six MNIST test images show the input format used by the network: grayscale 28x28 images with bright digits on a dark background.

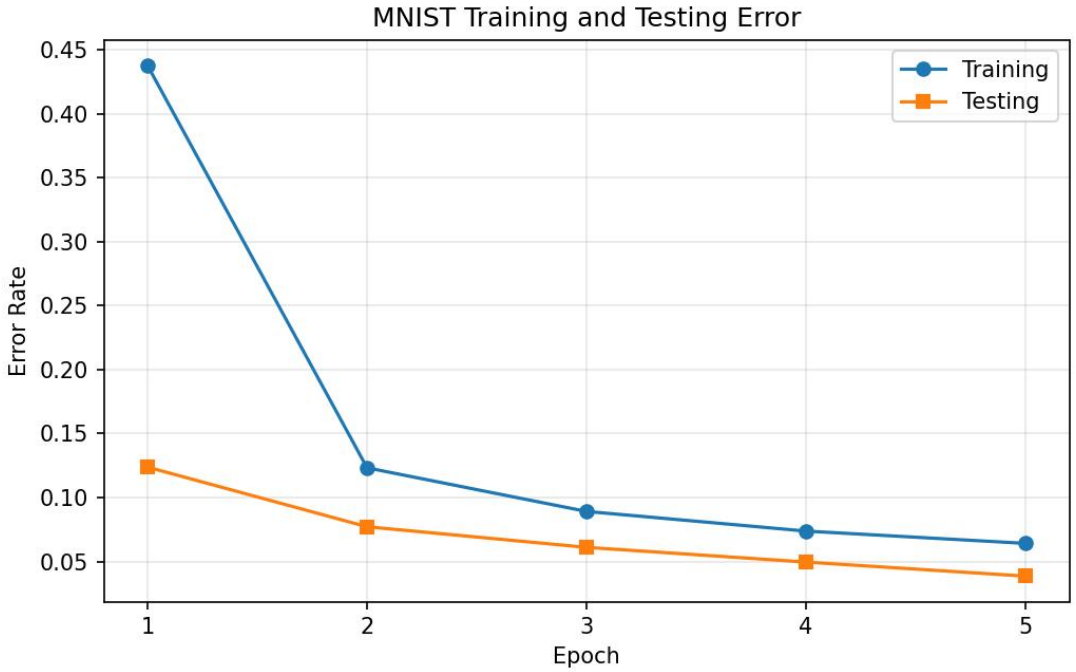


The CNN used the required structure: two convolution layers, max pooling, dropout, a 50-node fully connected layer, and a final 10-class output layer using log softmax.



The training and testing curves show that the model learned the MNIST digit task successfully over five epochs. The testing accuracy increased as training progressed, while the error decreased.





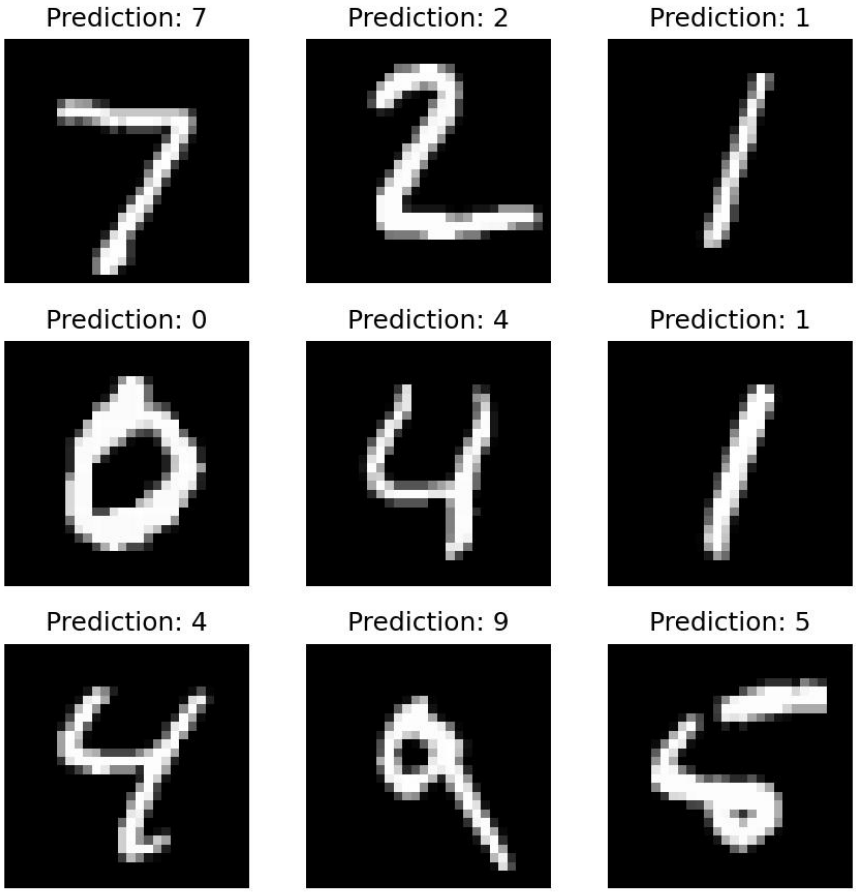
The trained network was saved as `mnist_cnn.pt` so it could be reused in later tasks.

After reloading the saved model and setting it to evaluation mode, the network correctly classified all first ten examples from the MNIST test set.

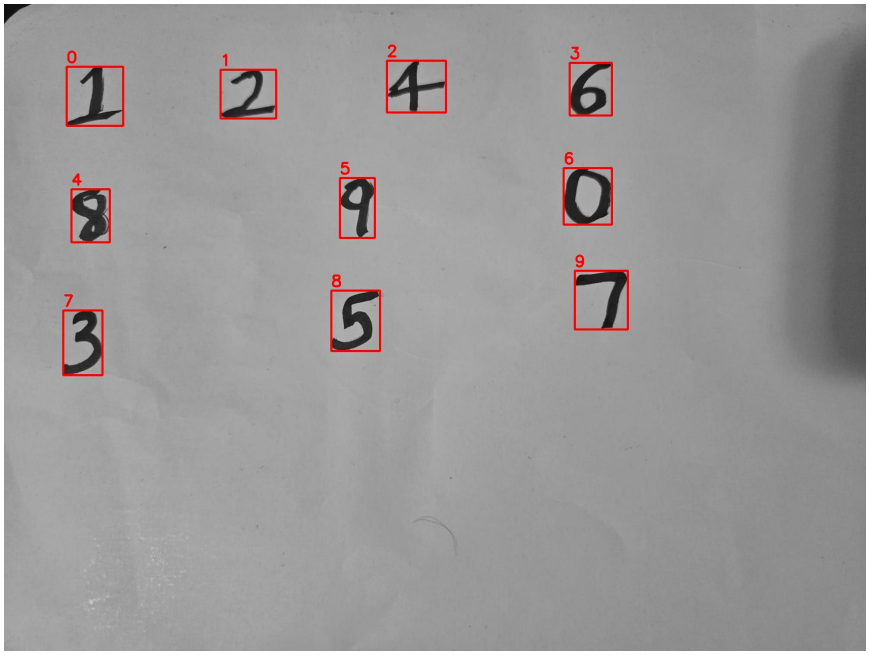
Example	Network Outputs for Digits 0–9	Prediction	Correct Label
0	−16.03, −16.54, −10.61, −10.19, −16.22, −19.07, −27.85, −0.00, −15.36, −9.61	7	7
1	−6.94, −8.20, −0.02, −8.17, −19.00, −10.44, −5.35, −18.25, −4.11, −21.70	2	2
2	−11.53, −0.00, −6.97, −8.43, −8.09, −9.88, −9.00, −6.58, −6.99, −9.48	1	1
3	−0.00, −21.66, −9.94, −13.96, −18.60, −9.53, −11.28, −10.25, −14.25, −12.80	0	0
4	−15.09, −16.90, −12.38, −13.97, −0.00, −13.22, −11.17, −8.87, −12.62, −6.33	4	4
5	−14.61, −0.00, −9.57, −8.91, −8.95, −12.77, −13.39, −6.12, −8.80, −10.23	1	1
6	−18.41, −10.83, −13.42, −10.50, −0.02, −8.75, −14.76, −4.99, −5.89, −4.96	4	4
7	−16.06, −7.62, −8.38, −5.37, −2.47, −6.81, −11.93, −6.52, −4.40, −0.11	9	9
8	−7.96, −12.73, −7.11, −13.68, −8.56, −0.20, −1.75, −15.39, −5.26, −10.07	5	5

Example	Network Outputs for Digits 0 - 9	Prediction	Correct Label
9	-12.93, -16.44, -12.94, -9.51, -5.90, -11.21, -17.78, -2.41, -5.94, -0.10	9	9

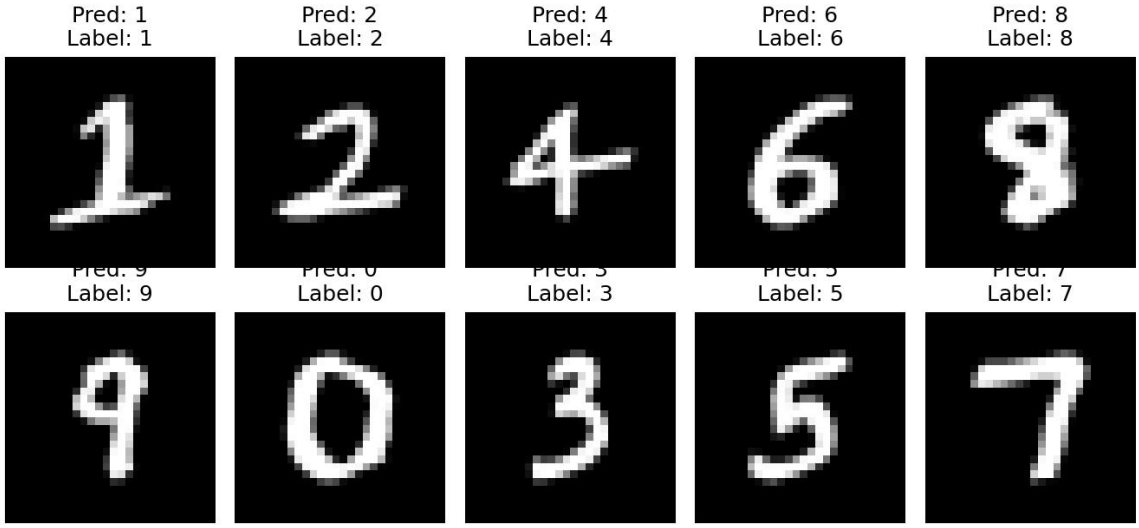
The plot below shows the first nine MNIST test images with the predicted class above each image.



I tested the saved MNIST network on my own handwritten digit sheet. The original image was rotated, thresholded, cropped into individual digits, resized to 28x28, and inverted to match the MNIST white-on-black intensity pattern.



The network correctly classified all ten handwritten digits from the sheet.

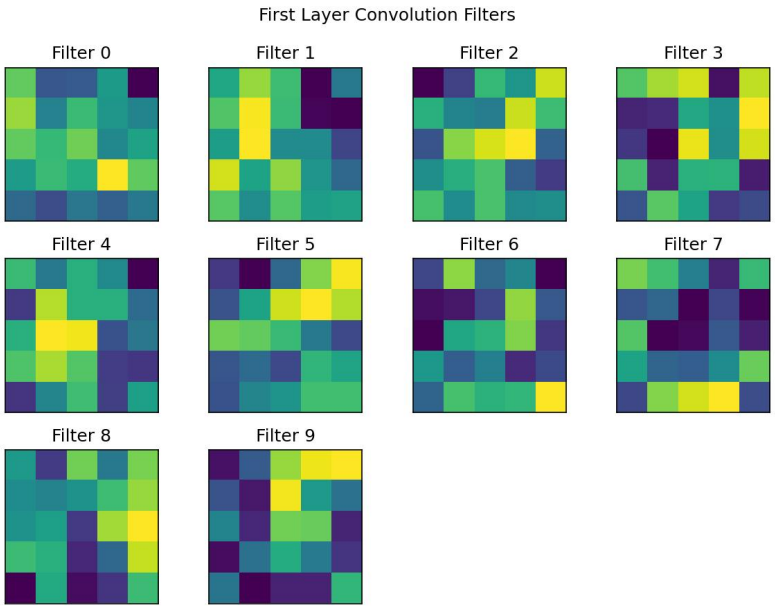


Index	Label	Prediction	Correct?
0	1	1	Yes
1	2	2	Yes
2	4	4	Yes
3	6	6	Yes
4	8	8	Yes
5	9	9	Yes
6	0	0	Yes
7	3	3	Yes
8	5	5	Yes

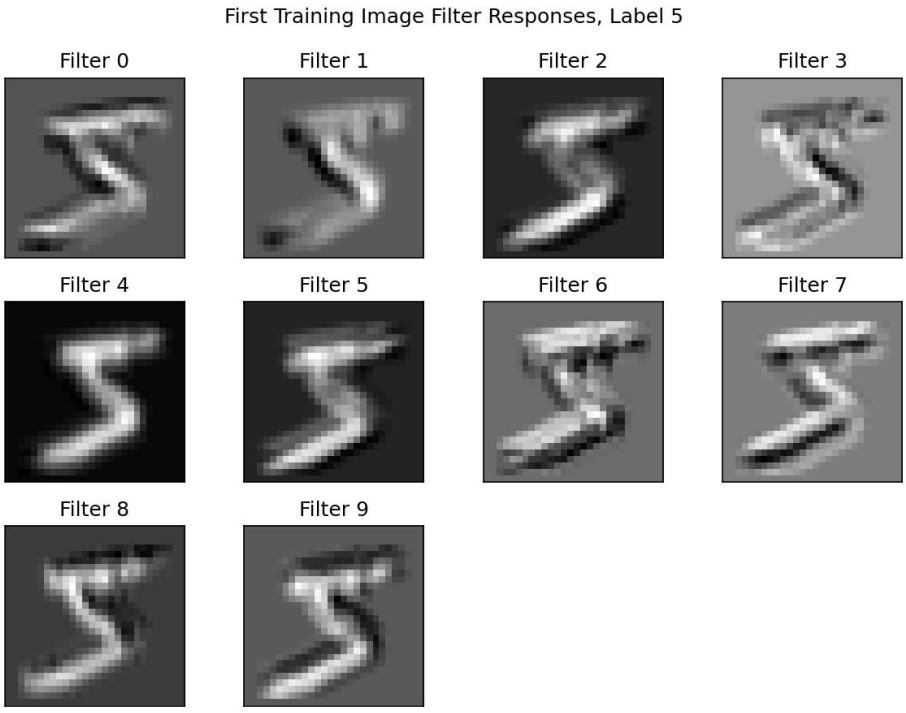
```
Index Label Prediction Correct?
    9     7          7    Yes
```

Task 2: Examining the network

The printed model showed that the first layer was named `conv1` and had weight shape 10 x 1 x 5 x 5. Each of the ten filters learned a different small stroke or contrast pattern.



Applying the ten first-layer filters to the first training example shows that the filters respond to different parts of the digit, such as edges, curves, and local bright-dark transitions.

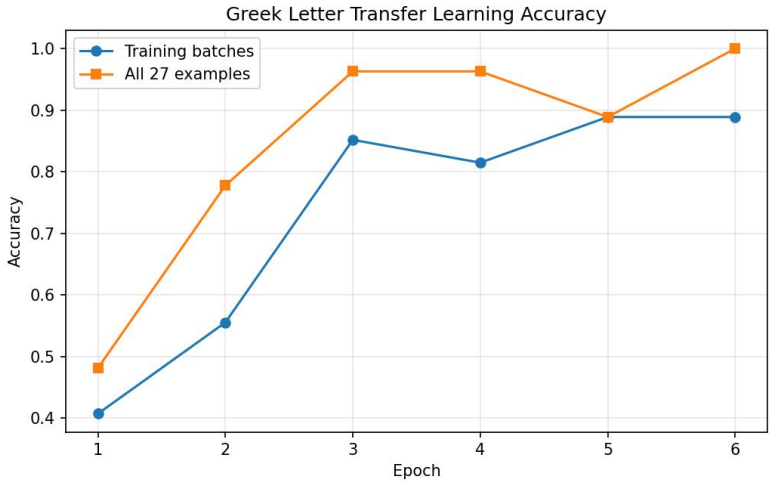


Task 3: Transfer Learning on Greek Letters

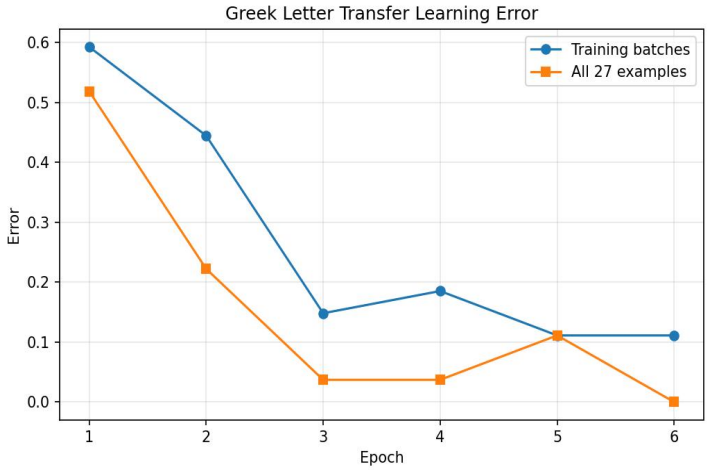
For Greek letter recognition, I reused the trained MNIST network, froze the pretrained weights, and replaced the final classifier with a three-output layer for alpha, beta, and gamma. The dataset contained 27 training examples. The model reached 100% accuracy on the 27 examples at epoch 6.

Modified network printout:

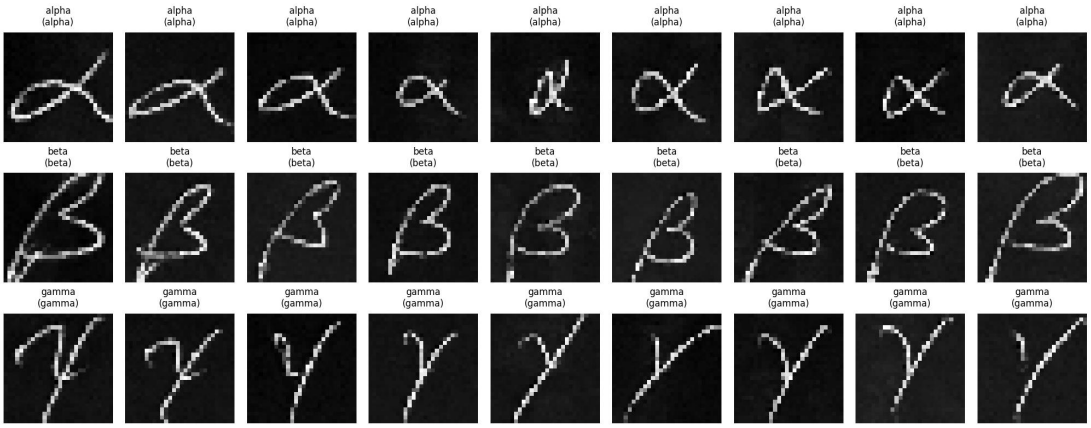
```
MyNetwork(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1))
  (conv2): Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))
  (dropout): Dropout2d(p=0.5, inplace=False)
  (fc1): Linear(in_features=320, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=3, bias=True)
)
```



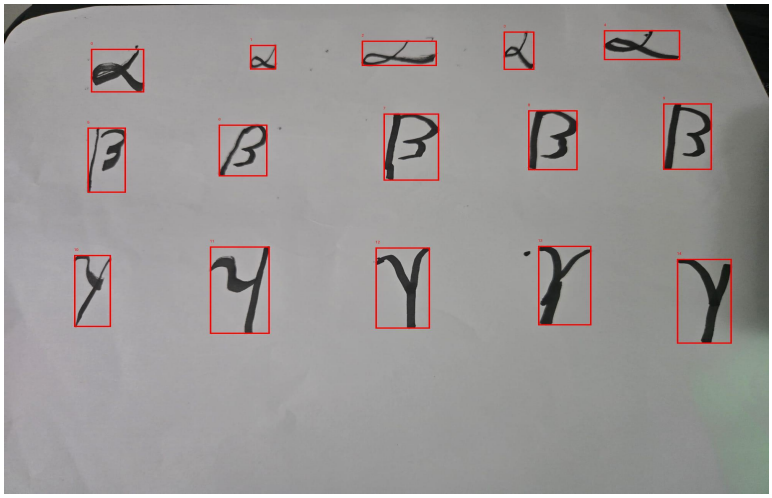
The training error plot shows the error falling as the replacement classifier learned the Greek-letter categories. The evaluation error reached 0.0 at epoch 6 on the 27 provided Greek examples.



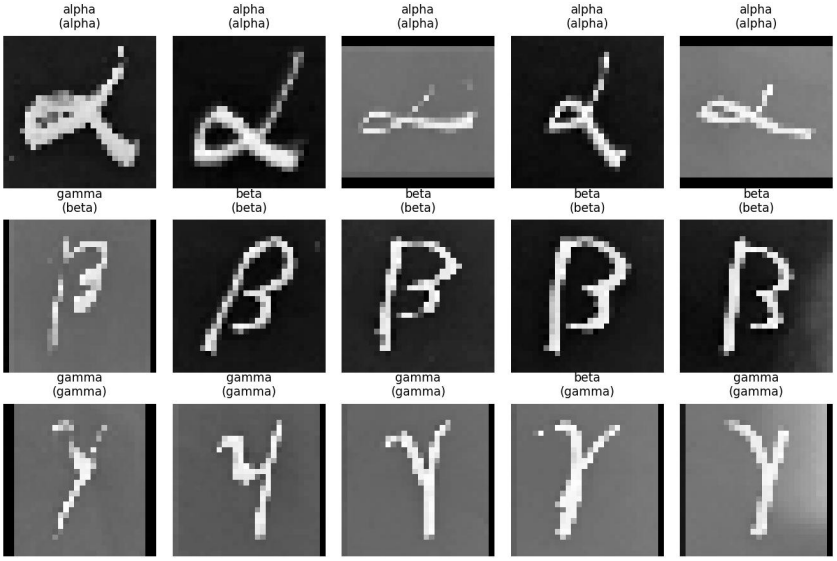
The prediction grid shows the trained transfer-learning model classifying the Greek letter examples.



I also tested the Greek transfer model on my own handwritten alpha, beta, and gamma symbols. The symbols were photographed on one sheet, automatically cropped into 128x128 images, and placed into alpha, beta, and gamma folders.



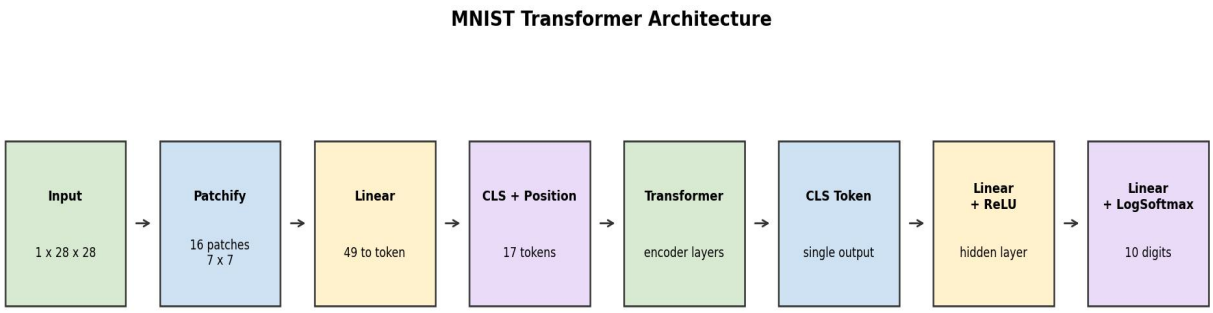
The model correctly classified 13 of the 15 custom handwritten Greek symbols. It correctly classified all five alpha examples, four of five beta examples, and four of five gamma examples. The first beta was predicted as gamma, and the fourth gamma was predicted as beta.



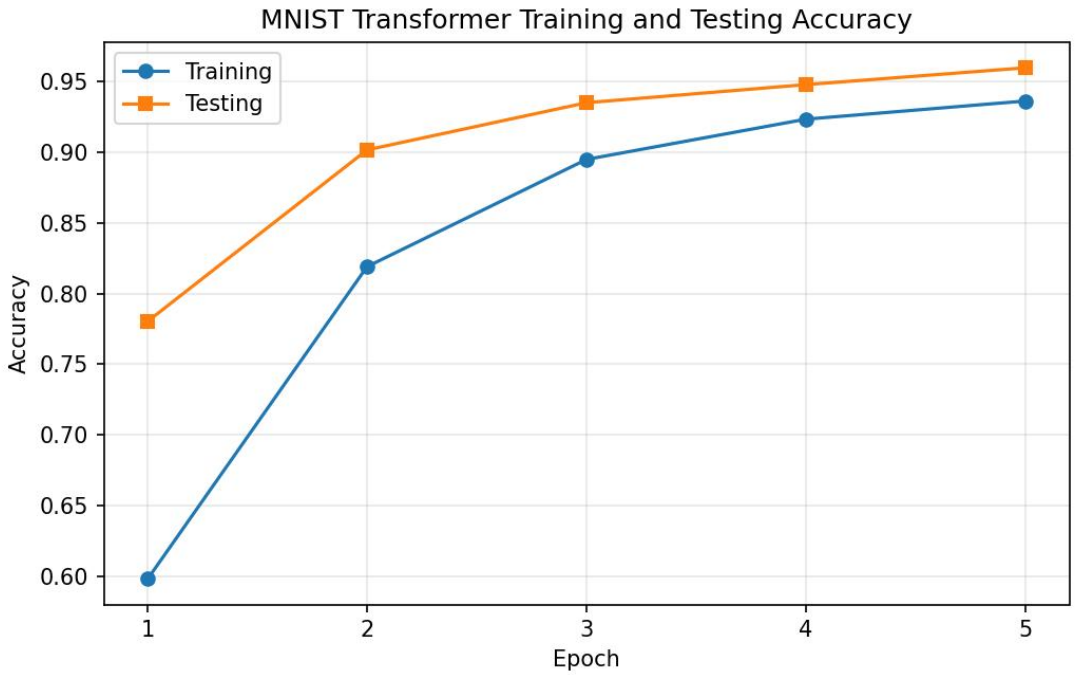
File	Prediction	Expected	Correct?
alpha_01.jpg	alpha	alpha	Yes
alpha_02.jpg	alpha	alpha	Yes
alpha_03.jpg	alpha	alpha	Yes
alpha_04.jpg	alpha	alpha	Yes
alpha_05.jpg	alpha	alpha	Yes
beta_01.jpg	gamma	beta	No
beta_02.jpg	beta	beta	Yes
beta_03.jpg	beta	beta	Yes
beta_04.jpg	beta	beta	Yes
beta_05.jpg	beta	beta	Yes
gamma_01.jpg	gamma	gamma	Yes
gamma_02.jpg	gamma	gamma	Yes
gamma_03.jpg	gamma	gamma	Yes
gamma_04.jpg	beta	gamma	No
gamma_05.jpg	gamma	gamma	Yes

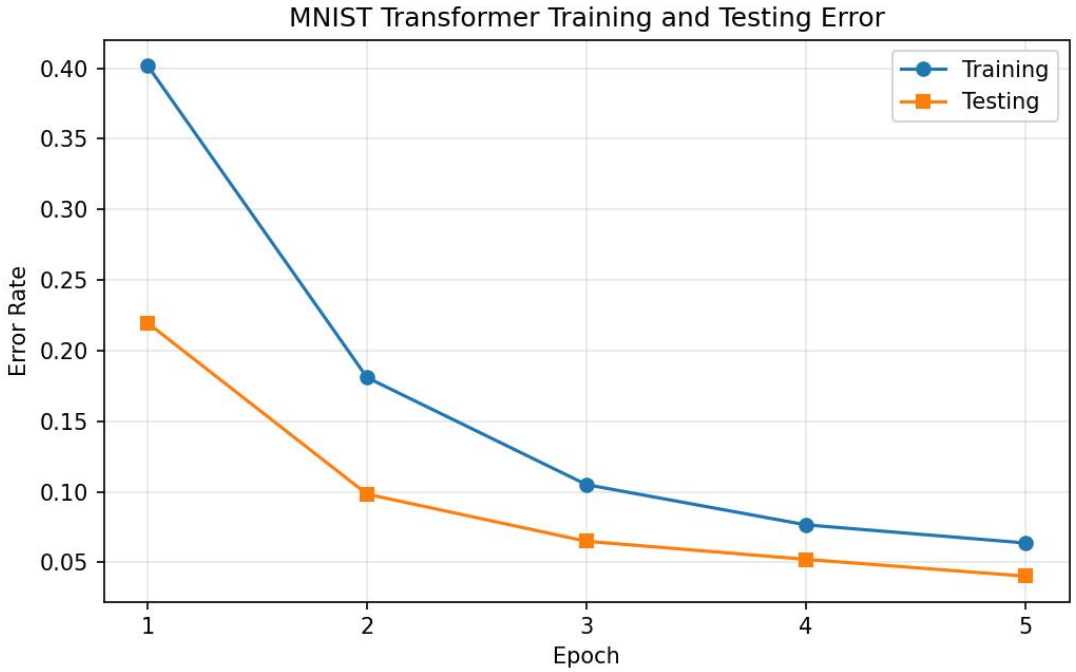
Task 4: Transformer Reimplementation

For Greek letter recognition, I reused the trained MNIST network, froze the pretrained weights, and replaced the final classifier with a three-output layer for alpha, beta, and gamma. The dataset contained 27 training examples. The model reached 100% accuracy on the 27 examples at epoch 6.



After five epochs, the transformer model reached 95.98% test accuracy.





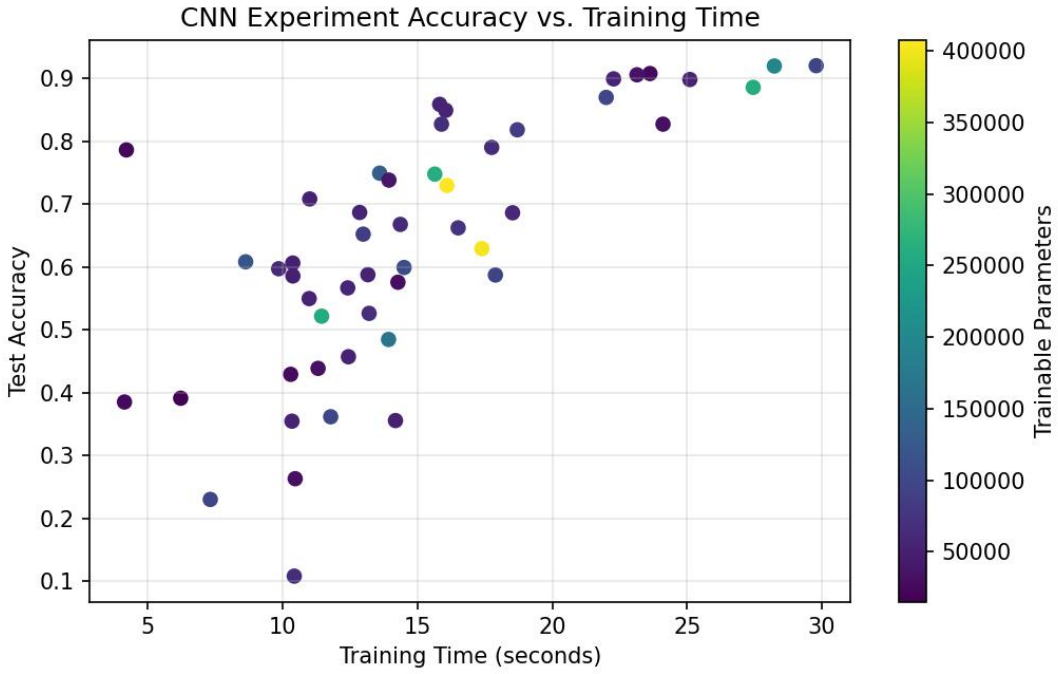
Task 5: Final Experiment: CNN Architecture Search

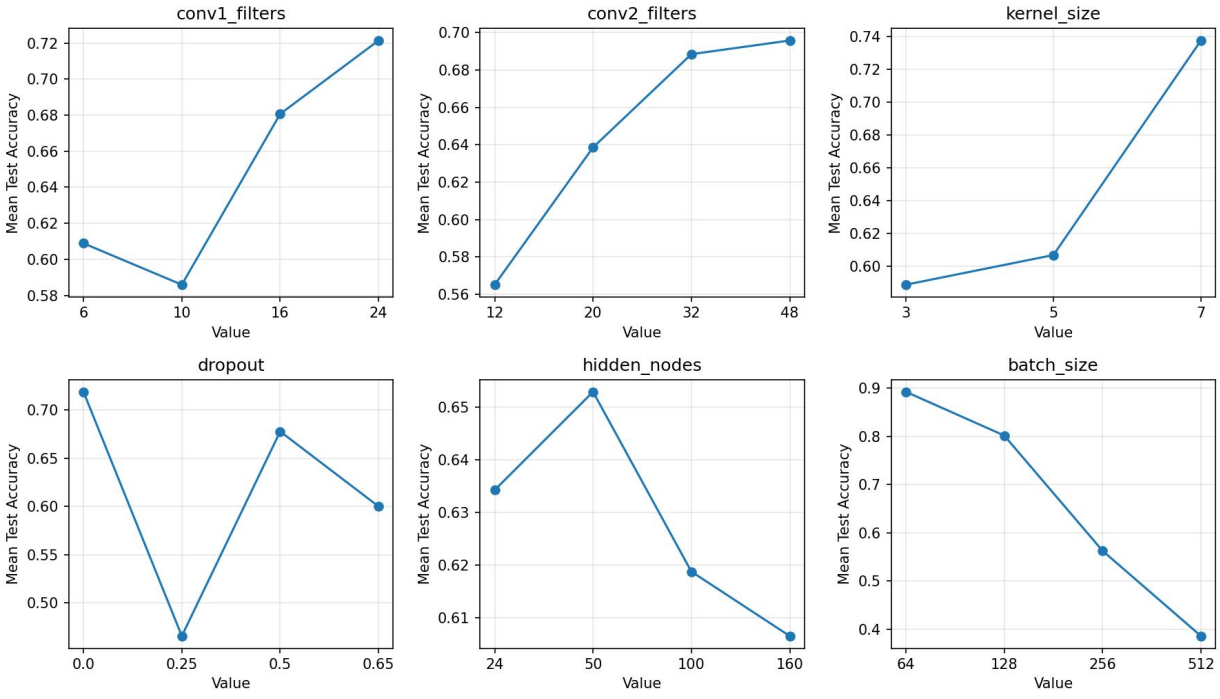
For the final experiment, I evaluated 50 CNN variations while changing convolution filter counts, kernel size, dropout rate, hidden-layer size, and batch size. The goal was to compare accuracy and training time rather than only train one fixed model.

Hypotheses:

- More convolution filters would improve accuracy but increase training time.
- A 5x5 kernel would work well because it matched the original model.
- Moderate dropout would generalize better than no dropout, but very high dropout would slow learning.
- More hidden nodes would help until the classifier had enough capacity.
- Smaller batch sizes would likely give better accuracy for a fixed number of epochs.

The best test accuracy was 92.00% in run 31, using 16 first-layer filters, 48 second-layer filters, a 7x7 kernel, 0.65 dropout, 24 hidden nodes, and batch size 64. The fastest run above 90% accuracy was run 24, which reached 90.55% accuracy.



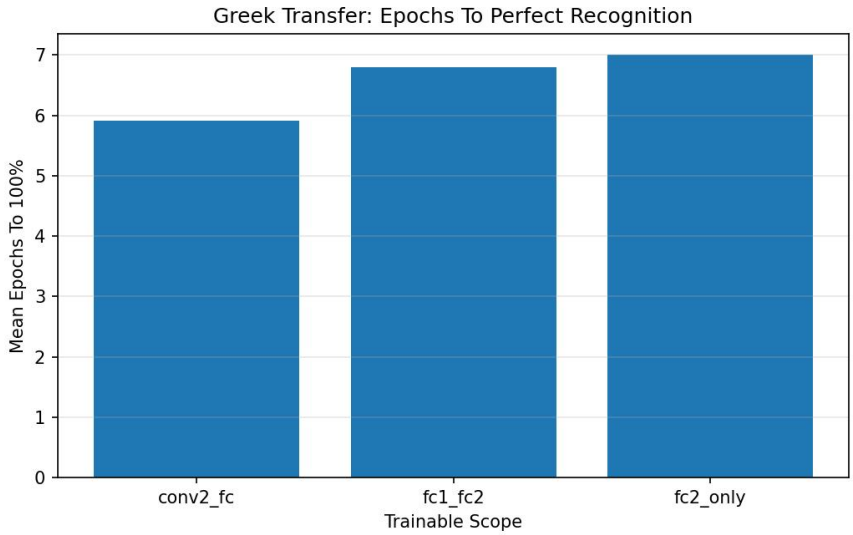


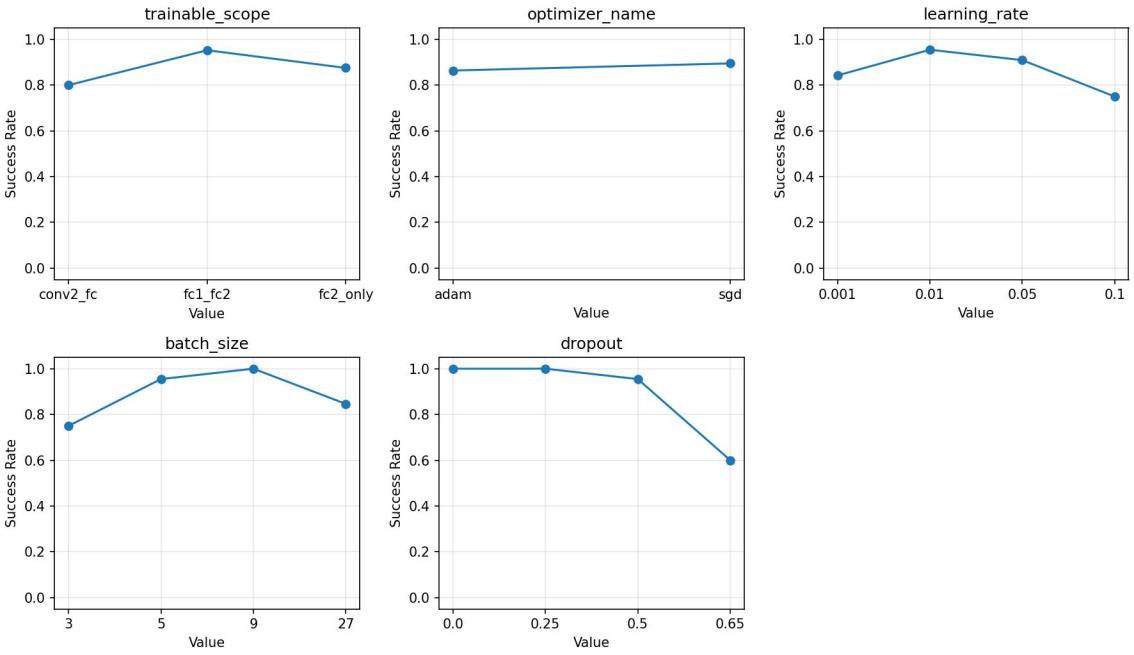
The results supported the filter-count and batch-size hypotheses. Larger convolution layers generally improved accuracy, and batch size 64 had the best average accuracy. The kernel-size hypothesis was not supported because 7x7 filters performed best in this experiment. Dropout had mixed results because it interacted strongly with model size and batch size.

Extensions:

Greek Transfer Learning Extension:

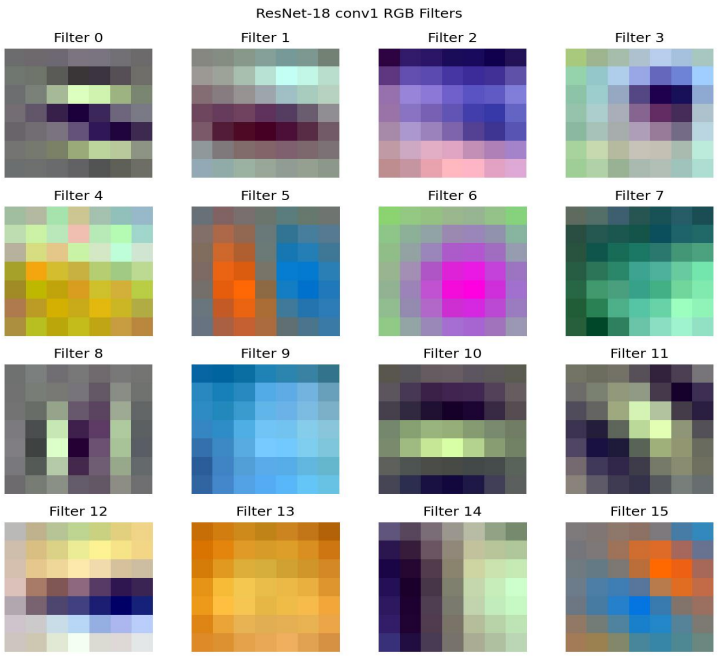
I extended the Greek transfer task by evaluating 60 variations across trainable scope, optimizer, learning rate, batch size, and dropout. The best configuration reached 100% accuracy in one epoch by unfreezing the second convolution layer and classifier, using SGD, learning rate 0.01, batch size 5, and dropout 0.25.

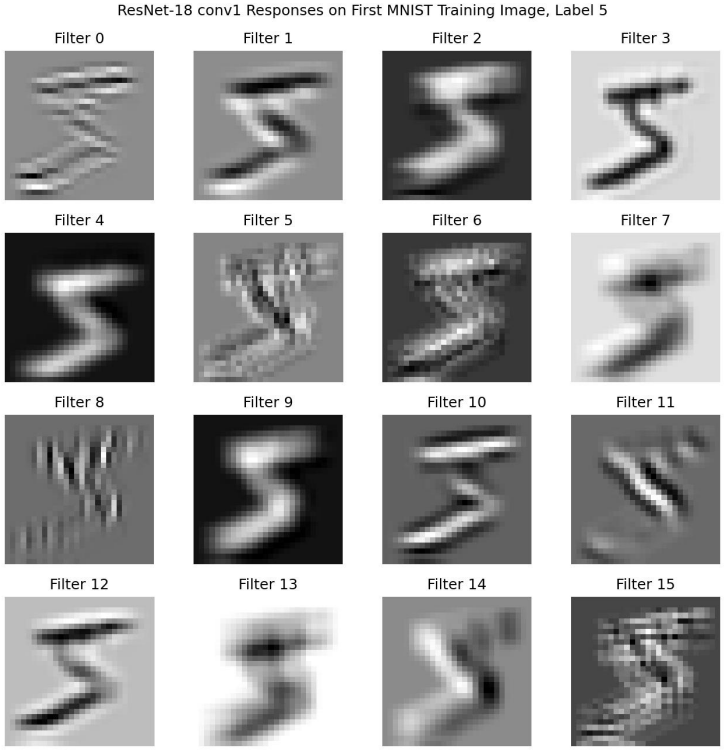


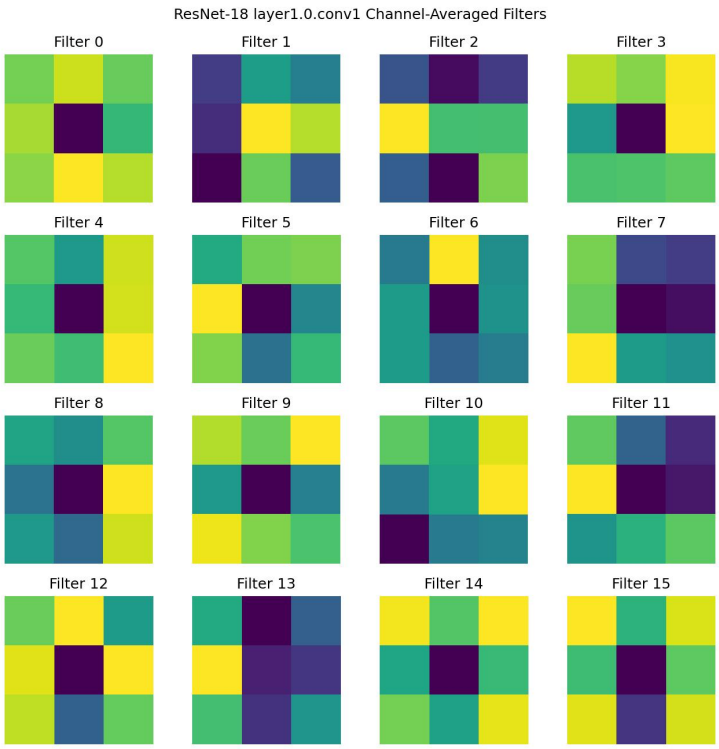


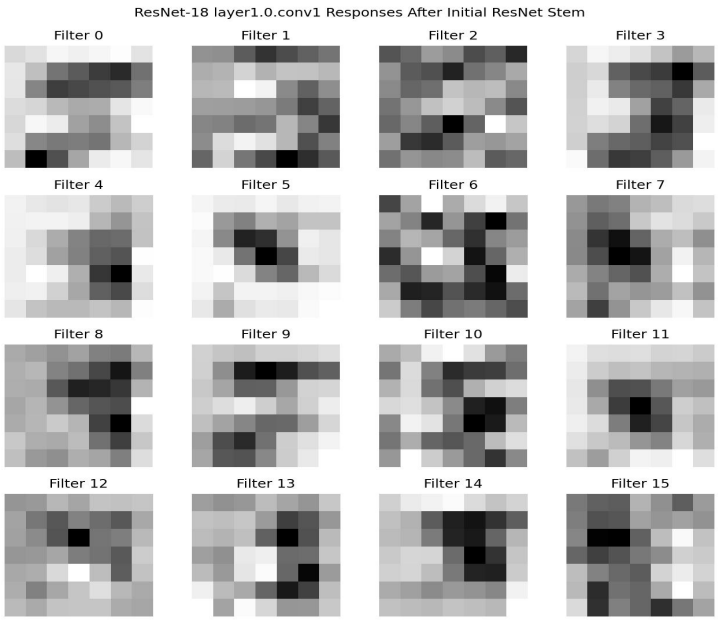
Pretrained ResNet Filter Analysis:

I loaded a pretrained torchvision ResNet-18 and visualized filters from its first convolutional layer and an early layer inside `layer1`. Compared with the small MNIST CNN, the pretrained ResNet filters are larger, color-aware, and more varied because they were trained on natural images.


ResNet-18 conv1 RGB Filters
Filter 0
Filter 1
Filter 2
Filter 3
Filter 4
Filter 5
Filter 6
Filter 7
Filter 8
Filter 9
Filter 10
Filter 11
Filter 12
Filter 13
Filter 14
Filter 15

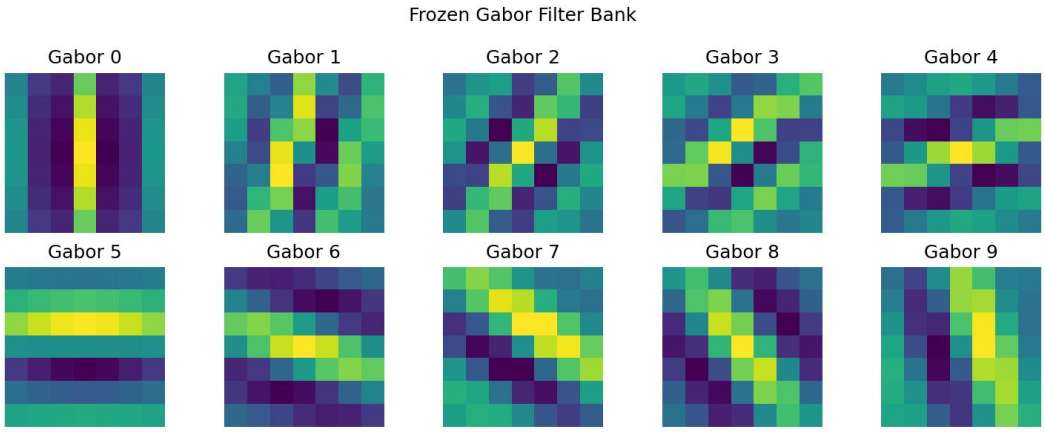

ResNet-18 conv1 Responses on First MNIST Training Image, Label 5
Filter 0
Filter 1
Filter 2
Filter 3
Filter 4
Filter 5
Filter 6
Filter 7
Filter 8
Filter 9
Filter 10
Filter 11
Filter 12
Filter 13
Filter 14
Filter 15

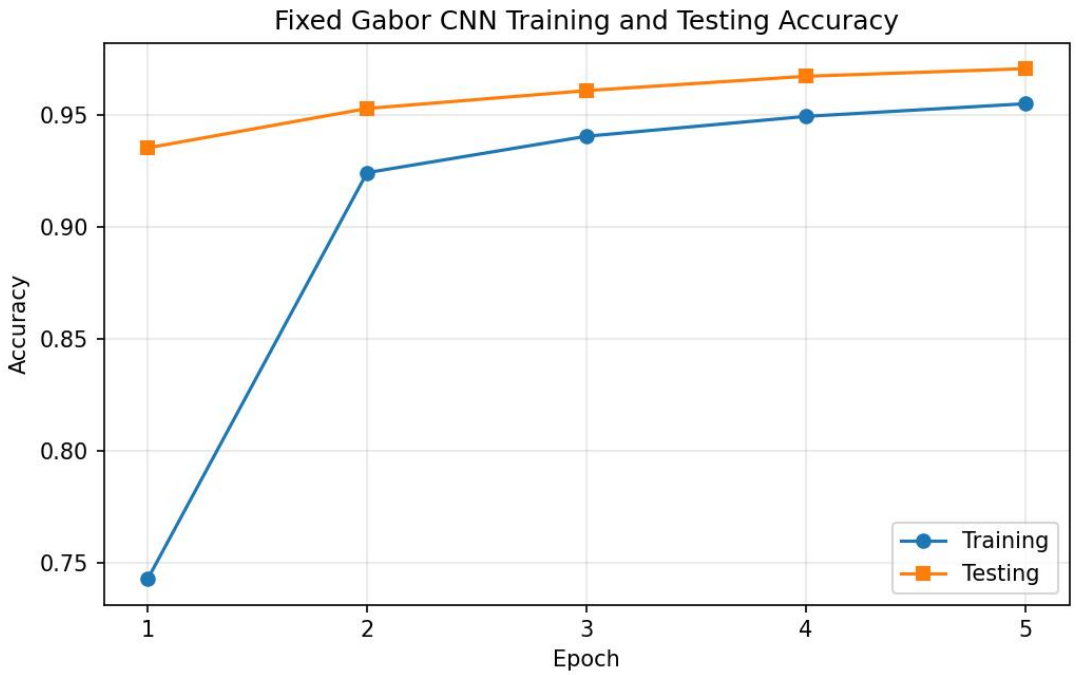

ResNet-18 layer1.0.conv1 Channel-Averaged Filters
Filter 0
Filter 1
Filter 2
Filter 3
Filter 4
Filter 5
Filter 6
Filter 7
Filter 8
Filter 9
Filter 10
Filter 11
Filter 12
Filter 13
Filter 14
Filter 15


ResNet-18 layer1.0.conv1 Responses After Initial ResNet Stem
Filter 0
Filter 1
Filter 2
Filter 3
Filter 4
Filter 5
Filter 6
Filter 7
Filter 8
Filter 9
Filter 10
Filter 11
Filter 12
Filter 13
Filter 14
Filter 15

Fixed Gabor First Layer:

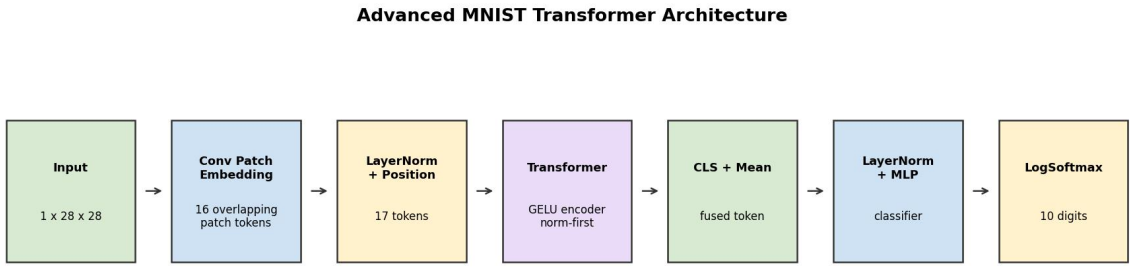
I replaced the first convolution layer of the MNIST CNN with a frozen bank of hand-designed Gabor filters. This model reached 97.09% test accuracy after five epochs, which was higher than the original CNN run. This suggests that edge- and stroke-oriented filters are a useful prior for MNIST.

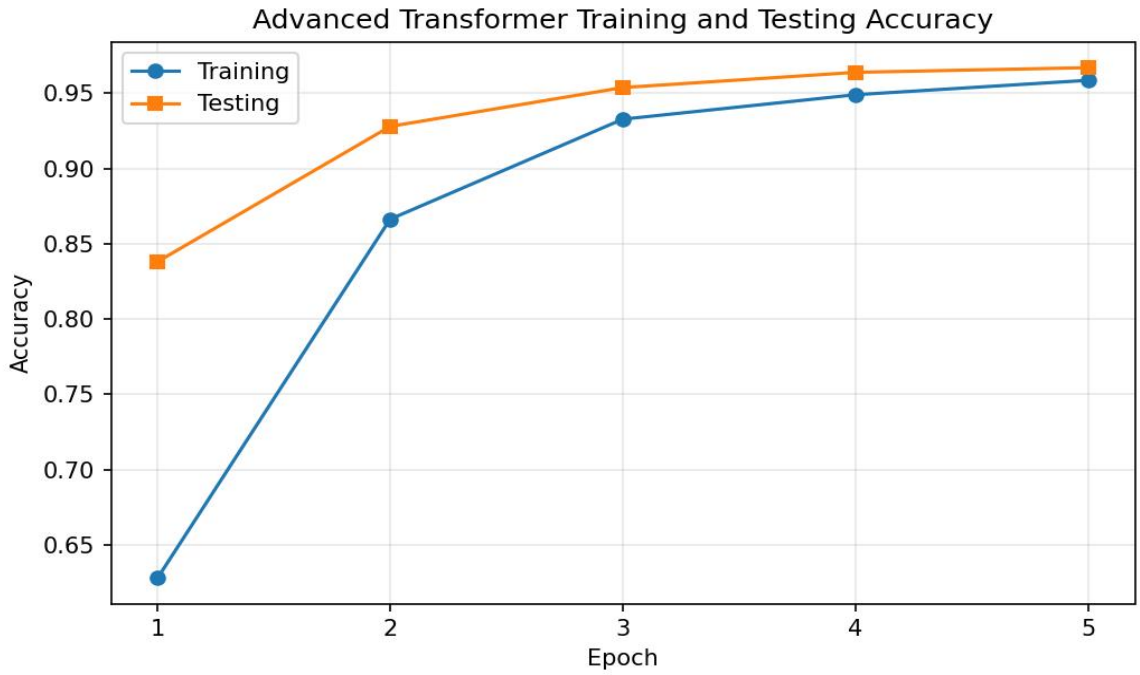




Advanced Transformer:

I improved the transformer model by using a convolutional patch embedding block and a classifier that combined the CLS token with the mean token representation. The advanced transformer reached 96.69% test accuracy after five epochs, improving over the simpler transformer.





Reflection:

This project helped me understand how the same recognition problem can be approached through several network designs. The CNN performed well because its convolution filters naturally match local image features such as strokes and edges. Examining the filters made the network feel less like a black box because I could see what the first layer was detecting. Transfer learning also showed how useful pretrained features can be, even when the new dataset is very small. The architecture experiments were especially useful because they showed that model performance depends on interactions between design choices, not just one parameter at a time.

Acknowledgements:

I consulted the course project instructions, PyTorch and torchvision documentation, and the provided Greek letter dataset. I also used examples from PyTorch tutorials as references for loading datasets, defining models, training networks, and saving model weights.